

Opinion Mining in NLP

Opinion Mining in NLP

Opinion mining, a key subfield of natural language processing (NLP), focuses on discerning the sentiment behind textual content. This process categorizes text as positive, negative, or neutral and has wide-ranging applications, from customer review analysis to market research and social media monitoring.

Opinion Mining Techniques

Each opinion mining technique offers a unique approach to extracting sentiment from textual data.

1. **Sentiment Polarity Classification**
2. **Emotion Detection**
3. **Aspect-based sentiment analysis**
4. **Multilingual Sentiment Analysis**

1. Sentiment Polarity Classification

- It provides a detailed sentiment score for a given text, ranging from negative to positive.
- This technique is invaluable for businesses analyzing customer reviews and social media posts.
- By assigning a sentiment score, businesses can make informed decisions based on the general sentiment of their customer base.

1. Sentiment Polarity Classification example

- To implement this technique, we will first import the ``pipeline`` module from the ``transformers`` library. Next, we will initialize the `sentiment analysis` pipeline using the `'nlptown/bert-base-multilingual-uncased-sentiment'` model. Then, we will pass a sample text into this pipeline, which will return two variables: the first is ``label``, which will give one of the categories from the `label_map`, ranging from 1 star to 5 stars (1 star being negative, 3 stars being neutral, and 5 stars being positive). The second variable is ``score``, which represents the confidence percentage. Finally, we will format the output and print the results.

1. Sentiment Polarity Classification example-2

```
from transformers import pipeline

sentiment_pipeline = pipeline('sentiment-analysis', model='nlpTown/bert-base-multilingual-uncased-sentiment')

text = "The movie was absolutely fantastic! I loved every moment of it."

result = sentiment_pipeline(text)[0]

label_map = {
    '1 star': 'Very Negative',
    '2 stars': 'Negative',
    '3 stars': 'Neutral',
    '4 stars': 'Positive',
    '5 stars': 'Very Positive'
}

custom_label = label_map[result['label']]
confidence_percentage = result['score'] * 100

print(f"Sentiment: {custom_label}")
print(f"Confidence: {confidence_percentage:.2f}%")
```

Output:

Sentiment: Very Positive
Confidence: 95.22%

2. Emotion Detection

- Emotion detection is an advanced version of the above technique, which identifies the emotion expressed in the text, such as joy, anger, sadness, etc.
- This technique is particularly helpful in understanding the feelings and attitudes of the individuals behind the text.
- It enables businesses to comprehend customer feedback more effectively than the previous technique.

Example

To implement this technique, we will first import the `pipeline` module from the transformers library, then initialize the emotion detection pipeline using the 'j-hartmann/emotion-english-distilroberta-base' model. Next, we will call this pipeline with a sample text, which will return a label indicating the emotion and a score variable giving the confidence percentage. Finally, we will format the output and print it.

```
from transformers import pipeline

emotion_pipeline = pipeline('text-classification', model='j-hartmann/emotion-english-distilroberta-base')

text = "I am so excited about the new project!"

result = emotion_pipeline(text)[0]

custom_label = result['label'].capitalize()
confidence_percentage = result['score'] * 100

print(f"Emotion: {custom_label}")
print(f"Confidence: {confidence_percentage:.2f}%")
```

Output:

<https://www.geeksforgeeks.org/nlp/opinion-mining-in-nlp/>

Emotion: Joy
Confidence: 97.58%

3. Aspect-based sentiment analysis

This technique addresses this issue by providing the sentiment based on **specific aspects (topics)** within the text.

This allows businesses to pinpoint areas for improvement and understand customer preferences in greater detail.

Example

To implement this technique, we will first import necessary libraries like spaCy and NLTK, and modules like `Matcher` and `SentimentIntensityAnalyzer` from spaCy and NLTK, respectively. Next, we will download the VADER lexicon and load the spaCy model, then define a sample text. We will initialize the `Matcher` and use the `SentimentIntensityAnalyzer` to analyze the text.

```
import spacy
from spacy.matcher import Matcher
from nltk.sentiment import SentimentIntensityAnalyzer
import nltk

nltk.download('vader_lexicon')

nlp = spacy.load("en_core_web_sm")

text = "The food was delicious, but the service was slow."

doc = nlp(text)

matcher = Matcher(nlp.vocab)
pattern = [{"LOWER": {"IN": ["food", "service"]}}]
matcher.add("AspectMatcher", [pattern])
```

```
sia = SentimentIntensityAnalyzer()

aspects = []

for match_id, start, end in matcher(doc):
    aspect = doc[start:end].text
    sentiment = None

    scores = sia.polarity_scores(aspect)
    sentiment = "positive" if scores["compound"] > 0 else "negative"

    aspects.append({"aspect": aspect, "sentiment": sentiment})

print(aspects)
```

Output:

<https://www.geeksforgeeks.org/nlp/opinion-mining-in-nlp/>

```
[{'aspect': 'food', 'sentiment': 'negative'}, {'aspect': 'service', 'sentiment': 'negative'}]
```

4. Multilingual Sentiment Analysis متعدد اللغات

Multilingual sentiment analysis allows us to determine the sentiment of **text in any language**.

This technique is particularly useful for global businesses that receive customer feedback in **multiple languages**.

Using this technique, we can directly obtain the sentiment of all customer feedback at once, **without the need to translate it into English first before performing sentiment analysis**.

Example

To implement this, we will first import the `pipeline` module from the `transformers` library. Next, we will create a pipeline using the 'nlptown/bert-base-multilingual-uncased-sentiment' model. We will then create a list of texts in different languages and pass them to the pipeline using a for loop. Finally, we will format the output and print the results.

```
from transformers import pipeline

multilingual_sentiment_pipeline = pipeline('sentiment-analysis', model='nlptown/bert-base-multilingual-uncased-sentiment')

texts = [
    "सेवा उत्कृष्ट थी।", # Hindi: "The service was excellent."
    "సినిమా చాలా నిరాశపరిచింది." # Telugu: "The movie was very disappointing."
]

results = [multilingual_sentiment_pipeline(text) for text in texts]
for text, result in zip(texts, results):
    print(f"Text: {text}\nSentiment: {result}\n")
```

Output:

```
Text: सेवा उत्कृष्ट थी।
Sentiment: [{'label': '5 stars', 'score': 0.4839303195476532}]

Text: సినిమా చాలా నిరాశపరిచింది.
Sentiment: [{'label': '3 stars', 'score': 0.3825204372406006}]
```

<https://www.geeksforgeeks.org/nlp/opinion-mining-in-nlp/>

Regular Expressions

Regular Expression Characters

There are following different type of characters of a regular expression:

1. Metacharacters
2. Quantifier
3. Groups and Ranges
4. Escape Characters or character classes

1-Metacharacters

Metacharacters	Description	Example
^	This character is used to match an expression to its right at the start of a string.	^a is an expression match to the string which starts with 'a' such as " aab ", " a9c ", " apr ", " aaaaab ", etc.
\$	The \$sign is used to match an expression to its left at the end of a string.	r\$ is an expression match to a string which ends with r such as " aaabr ", " ar ", " r ", " aannn9r ", etc.
.	This character is used to match any single character in a string except the line terminator, i.e. /n.	b.x is an expression that match strings such as " bax ", " b9x ", " bar ".
 	It is used to match a particular character or a group of characters on either side. If the character on the left side is matched, then the right side's character is ignored.	A b is an expression which gives various strings, but each string contains either a or b .
\	It is used to escape a special character after this sign in a string.	
A	It is used to match the character 'A' in the string.	This expression matches those strings in which at least one-time A is present. Such strings are " Amcx ", " mnAr ", " mnopAx4 ".
Ab	It is used to match the substring 'ab' in the string.	This expression matches those strings in which ' Ab ' is present at least one time. Such strings are " Abcx ", " mnAb ", " mnopAbx4 ".

2-Quantifiers

The quantifiers are used in the regular expression for specifying the number of occurrences of a character.

2-Quantifiers-2

Charact ers	Description	Example
+	This character specifies an expression to its left for one or more times.	s+ is an expression which gives " s ", " ss ", " sss ", and so on.
?	This character specifies an expression to its left for 0 (Zero) or 1 (one)times.	aS? is an expression which gives either " a " or " as ", but not "ass" .
*	This character specifies an expression to its left for 0 or more times	Br* is an expression which gives " B ", " Br ", " Brr ", " Brrr ", and so on...
{x}	It specifies an expression to its left for only x times.	Mab{5} is an expression which gives the following string which contains 5 b's: " Mabbbbb "
{x, }	It specifies an expression to its left for x or more times.	Xb{3, } is an expression which gives various strings containing at least 3 b's. Such strings are " Xbbb ", " Xbbbb ", and so on.
{x,y}	It specifies an expression to its left, at least x times but less than y times.	Pr{3,6}a is an expression which provides two strings. Both strings are as follows: " Prrrr " and " Prrrrr "

3-Groups and Ranges

The groups and ranges in the regular expression define the **collection of characters enclosed** in the brackets.

3-Groups and Ranges-2

Characters	Description	Example
()	It is used to match everything which is in the simple bracket.	A(xy) is an expression which matches with the following string: "Axy"
{ }	It is used to match a particular number of occurrences defined in the curly bracket for its left string.	xz{4,6} is an expression which matches with the following string: "xzzzzz"
[]	It is used to match any character from a range of characters defined in the square bracket.	xz[atp]r is an expression which matches with the following strings: "xzar", "xztr", and "xzpr"
[pqr]	It matches p, q, or r individually.	Following strings are matched with this expression: "p", "q", and "r".
[pqr][xy]	It matches p, q, or r, followed by either x or y.	Following strings are matched with this expression: "px", "qx", and "rx", "py", "qy", and "ry".
(?: ...)	It is used for matching a non-capturing group.	A(?:nt pple) is an expression which matches to the following string: "Apple"
[^.....]	It matches a character which is not defined in the square bracket.	Suppose, Ab[^pqr] is an expression which matches only the following string: "Ab"

3-Groups and Ranges-3

Characters	Description	Example
[a-z]	It matches letters of a small case from a to z.	This expression matches the strings such as: "a", "python", "good" .
[A-Z]	It matches letters of an upper case from A to Z.	This expression matches the strings such as: "EXCELLENT", "NATURE" .
^[a-zA-Z]	It is used to match the string, which is either starts with a small case or upper-case letter.	This expression matches the strings such as: "A854xb", "pv4fv", "cdlux" .
[0-9]	It matches a digit from 0 to 9.	This expression matches the strings such as: "9845", "54455"
[aeiou]	This square bracket only matches the small case vowels.	-
[AEIOU]	This square bracket only matches the upper-case vowels.	-
ab[^4-9]	It matches those digits or characters which are not defined in the square bracket.	This expression matches those strings which do not contain 5, 6, 7, and 8.

4-Escape Characters or Character Classes

Characters	Description
\s	It is used to match a one white space character.
\S	It is used to match one non-white space character.
\0	It is used to match a NULL character.
\a	It is used to match a bell or alarm.
\d	It is used to match one decimal digit, which means from 0 to 9.
\D	It is used to match any non-decimal digit.
\n	It helps a user to match a new line.
\w	It is used to match the alphanumeric [0-9a-zA-Z] characters.
\W	It is used to match one non-word character
\b	It is used to match a word boundary.

Examples for regulate expressions

Regex	Matches any string that
<code>hello</code>	contains {hello}
<code>gray grey</code>	contains {gray, grey}
<code>gr(a e)y</code>	contains {gray, grey}
<code>gr[ae]y</code>	contains {gray, grey}
<code>b[aeiou]bble</code>	contains {babble, bebble, bibble, bobble, bubble}
<code>[b-chm-pP]at ot</code>	contains {bat, cat, hat, mat, nat, oat, pat, Pat, ot}
<code>colou?r</code>	contains {color, colour}
<code>rege(x(es)? xps?)</code>	contains {regex, regexes, regexp, regexps}

Examples for regulate expressions-2

Regex	Matches any string that
<code>go*gle</code>	contains {ggle, gogle, google, gooogle, goooogle, ...}
<code>go+gle</code>	contains {gogle, google, gooogle, goooogle, ...}
<code>g(oog)+le</code>	contains {google, googoogle, googoogoogle, googoogoogoogle, ...}
<code>z{3}</code>	contains {zzz}
<code>z{3,6}</code>	contains {zzz, zzzz, zzzzz, zzzzzz}
<code>z{3,}</code>	contains {zzz, zzzz, zzzzz, ...}
<code>[Bb]rainf**k</code>	contains {Brainf**k, brainf**k}
<code>\d</code>	contains {0,1,2,3,4,5,6,7,8,9}
<code>\d{5}(-\d{4})?</code>	contains a United States zip code
<code>1\d{10}</code>	contains an 11-digit string starting with a 1
<code>[2-9] [12]\d 3[0-6]</code>	contains an integer in the range 2..36 inclusive

Examples for regulate expressions-3

Regex	Matches any string that
<code>Hello\nworld</code>	contains Hello followed by a newline followed by world
<code>mi.....ft</code>	contains a nine-character (sub)string beginning with mi and ending with ft (Note: depending on context, the dot stands either for “any character at all” or “any character except a newline”.) Each dot is allowed to match a different character, so both microsoft and minecraft will match.
<code>\d+(\.\d\d)?</code>	contains a positive integer or a floating point number with exactly two characters after the decimal point.
<code>[^i*&2@]</code>	contains any character other than an i, asterisk, ampersand, 2, or at-sign.
<code>//[^\r\n]*[^\r\n]</code>	contains a Java or C# slash-slash comment
<code>^dog</code>	begins with "dog"
<code>dog\$</code>	ends with "dog"
<code>^dog\$</code>	is exactly "dog"

References

Malware Detection

<https://perception-point.io/guides/malware/10-malware-examples-and-6-world-famous-attacks/>

<https://www.techtarget.com/searchsecurity/tip/10-common-types-of-malware-attacks-and-how-to-prevent-them>

https://www.researchgate.net/profile/Imtithal-Saeed/publication/272238656_A_Survey_on_Malwares_and_Malware_Detection_Systems/links/566284c608ae192bbf8cf1a5/A-Survey-on-Malwares-and-Malware-Detection-Systems.pdf

<https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=8949524>

Password management

<https://www.spiceworks.com/it-security/identity-access-management/articles/what-is-password-management/>

<https://blog.uniqkey.eu/advanced-password-management-techniques/>

<https://www.geeksforgeeks.org/password-management-in-cyber-security/>

<https://www.beyondtrust.com/blog/entry/top-15-password-management-best-practices>

Key Management

https://www.splunk.com/en_us/blog/learn/key-management.html

<https://www.geeksforgeeks.org/easy-key-management-in-cryptography/>

Regular expressions

<https://www.geeksforgeeks.org/write-regular-expressions/>

Strong password validation

<https://formulashq.com/the-ultimate-guide-to-regex-for-password-validation/>

Implementing **Regex** (Regular Expressions)

1- **Setting Minimum and Maximum Length**

One **common requirement for password validation** is setting a **minimum** and **maximum length**. By **using regex**, you can easily enforce this criterion.

For example, to set a **minimum length of eight characters** and a maximum length of sixteen characters, you can use the following **regex pattern**:

```
^(?=.*[A-Za-z])(?=.*\d)[A-Za-z\d]{8,16}$.
```

This pattern ensures that the password contains at least one letter, one digit, and is between eight and sixteen characters in length.

Implementing **Regex** (Regular Expressions) for **Password Validation**

2- **Enforcing Character Types**

Another essential aspect of password validation is enforcing the use of different character types. Typically, passwords should include a [combination of uppercase letters](#), lowercase letters, digits, and special characters. Regex can be used to guarantee the presence of these character types.

For example, the pattern `^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^\w\s])\S{8,16}$` ensures that the password contains at least one lowercase letter, one uppercase letter, one digit, one special character, and is between eight and sixteen characters in length.

Implementing **Regex** (Regular Expressions) for **Password Validation**

3- **Regex for Alphanumeric Passwords**

When allowing only alphanumeric passwords, the pattern can be simplified.

For instance, the regex `^[a-zA-Z0-9]{8,16}$` ensures that the password consists of only alphanumeric characters and is between eight and sixteen characters in length.

This type of pattern is often used in scenarios where special characters are not allowed, but a mix of letters and numbers is still required for a secure password.

Implementing **Regex** (Regular Expressions) for **Password Validation**

4- **Regex for Special Characters in Passwords**

If you want to require the use of special characters in passwords,

the pattern `^(?=.*[!@#$%^&*])[a-zA-Z0-9!@#$%^&*]{8,16}$` can be used. This regex ensures that the password contains at least one special character, in addition to alphanumeric characters, and is between eight and sixteen characters long. Including special characters in passwords can significantly enhance security by increasing the complexity of the password, making it harder for attackers to guess or crack.

bot detection

What is bot detection?

Bot detection is the process of identifying bot traffic by distinguishing it from traffic generated by real people using a website.

As a result, bots that wish to avoid detection (and the consequences of bot detection, like IP bans) attempt to mimic human behavior. This has led to an ongoing arms race between bot developers and the people who create bot detection software.

Primitive bot detection methods (like CAPTCHA), don't always work against advanced bots, so people who develop bot detection tools need to create increasingly sophisticated solutions. For example, some bot detection tools now use machine learning to create behavioral models that learn and adapt to bots as they evolve, or implement algorithms that detect unusual or unexpected behavior.

Some of the techniques that bot detection tools

Some of the techniques that **bot detection** tools use includes:

1. Behavioral analysis
2. Browser and device fingerprinting
3. IP address analysis
4. Traffic analysis
5. CAPTCHA challenges

1- Behavioral analysis

Behavioral analysis focuses on analyzing **user interactions** and patterns to look for indications of bot-like behavior. It looks at factors like:

- Mouse movements
- Clicks
- Scrolling speed
- Typing patterns
- Navigation paths
- Time spent on page

Some **advanced bot detection methods** use **machine learning and AI** to develop **more sophisticated software that can learn and adapt** to the evolving behavior of bots. These models can analyze huge amounts of data to identify very subtle patterns and anomalies that distinguish bots from humans.

2- Browser and device fingerprinting

Browser fingerprinting is a method websites use to create a unique "fingerprint" of a visitor's browser and device configuration. This fingerprint is based on various attributes like:

- Browser type
- Version
- Operating system
- Screen resolution
- Installed fonts
- Plugins

Websites can also examine hardware characteristics like CPU, GPU, and network adapters to create a unique device fingerprint. They can use the device fingerprint in combination with a browser fingerprint to improve bot detection accuracy.

3- IP address analysis

Every device connected to the internet has a unique IP address. Websites can log your IP address when you access their content, which they can then use to [track and monitor your activity](#). They can use this information to analyze your IP address and determine whether you are a real human or a bot, by looking at:

- IP reputation on services that track the history of different IP addresses
- Multiple requests from the same IP address, which can indicate automated activity
- Geolocation anomalies, such as traffic originating from unexpected or unusual locations compared to the website's typical audience
- Databases of known bot networks or data centers associated with bot activity

4- Traffic analysis

Bot detection relies on traffic analysis to examine patterns in website traffic to identify large surges of bot traffic. Websites track a number of traffic metrics like page views, unique visitors, session duration, and traffic sources to identify anomalies that could indicate bot activity, such as:

- Sudden spikes in traffic
- High bounce rates
- Short session durations
- Suspicious traffic sources

Websites can analyze these patterns in real time, so they can take immediate action in the event of a suspected bot attack. (For example, DDoS attacks.)

5- CAPTCHA challenges

CAPTCHA works by presenting users with challenges that are easy for humans to solve but difficult for bots.

Websites will usually present these challenges once they have already detected bot-like activity, and want the user to prove they are human. They do this to avoid a situation where everyone visiting a website has to solve challenges before accessing content, as that would cause frustration among real human visitors.

However, many bots are able to [solve CAPTCHAs or even avoid them altogether](#) by mimicking human behavior (so they don't trigger them in the first place).

Bot detection in action (example)

If a user's interactions differ from typical human behavior by showing unusual or repetitive patterns, **it will raise suspicion of bot activity**. Some bot detection tools **implement algorithms that can detect unusual or unexpected behavior, even when bots try to mimic human patterns.**

For example, imagine a bot that's programmed to browse an ecommerce website, add items to its cart, and proceed to checkout. It may try to mimic human behavior by:

- Randomizing clicks and navigation - by clicking on different product pages at varying intervals, scrolling through product descriptions, and adding items to the cart randomly
- Varying session duration - by spending different amounts of time on each page and in the checkout process, mimicking human browsing habits
- Using a common user agent - disguising itself as a popular web browser that many real users use

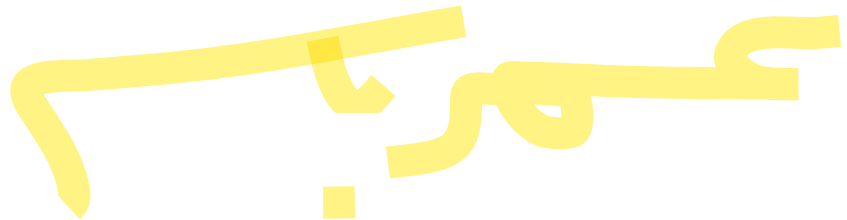
An anomaly detection algorithm that monitors the website's traffic can analyze various aspects of the bot's behavior and compare it to a database of normal patterns from real human interactions. Here's how it might identify the bot:

- Bots can click on more products and navigate through pages much **faster than a human**.
- Even with randomized actions, bots can **exhibit repetitive patterns** in its checkout process, such as always selecting the same shipping options or payment method.
- Bots can be **active outside usual shopping hours**, such as late at night or early in the morning, when human users are less likely to browse.
- While bots might use a common user agent, the algorithm could detect **inconsistencies in other technical details**, such as the browser version or the operating system they claim to be running on.

Bot detection in action (example)-2

Once the algorithm identifies a bot, the website could take action to prevent it from accessing the site:

- The bot could be presented with a CAPTCHA challenge to verify its humanness.
- The bot's access could be temporarily limited or slowed down to prevent it from overloading the website.
- The website might block the bot's IP address from accessing the website altogether.



How bot detection affects web scraping

- **Web scraping** is an automatic method to obtain large amounts of data from websites. Most of this data is unstructured data in an HTML format which is then converted into structured data in a spreadsheet or a database so that it can be used in various applications.
- **Bot detection** is the number one challenge facing anyone who wants to extract public data from websites. Websites use a number of techniques to identify and block automated bots, and that usually includes web scrapers. The techniques websites use can include:
 - IP blocking and blacklisting
 - CAPTCHA challenges
 - Rate limiting
 - Dynamic content changes

These techniques can throttle or entirely prevent scrapers from accessing a website, or – in the case of dynamic content changes – they can make it difficult for scrapers to consistently and reliably extract data.

An overview of CAPTCHA

A CAPTCHA test is used to identify whether an internet user is a human or a bot. CAPTCHA is a full form for "Completely Automated Public Turing Test to Distinguish Robots from People." On the Internet, CAPTCHA and reCAPTCHA tests are frequently encountered. Such experiments are one way to monitor bot migration, however the approach has certain drawbacks.

Despite the fact that CAPTCHAs are intended to impede mechanized bots, CAPTCHAs are themselves robotized. They're customized to spring up in specific parts on a site, and they consequently pass or bomb clients.

i'm a captcha!



i'm not a robot



CAPTCHA Advantages and Disadvantages

key Advantages of CAPTCHA:

- It's highly worthwhile to discourage spam and unwanted access
- It can stop fake and ill-intended website access or usage
- CAPTCHAs are an easy way to improve the website's accessibility
- For human minds, making sense of CAPTCHA is easy.

key disadvantages of CAPTCHA:

- Don't consider CAPTCHA as a foolproof tool as it's not. It's most effective for spam. The rest of its applications are flawed and highly restricted.
- Solving CAPTCHA needs time and effort, which is not feasible for every website user. Some might even drop the idea of visiting a website if it uses CAPTCHA. Hence, it might lead to a certain drop in website traffic. If the sole purpose of having a website is to generate leads and traffic, CAPTCHA shouldn't be used.
- CAPTCHAs are not suitable for visually impaired users. So, you won't be able to target such users if you use CAPTCHA.

Examples of type CAPTCHA

1-Text CAPTCHAs

The most standard kind of check is text CAPTCHAs. These CAPTCHAs can fuse prominent articulations or explanations, similarly as uncommon digits and letters blends. Some substance-based CAPTCHAs break down different kinds of capitalization.

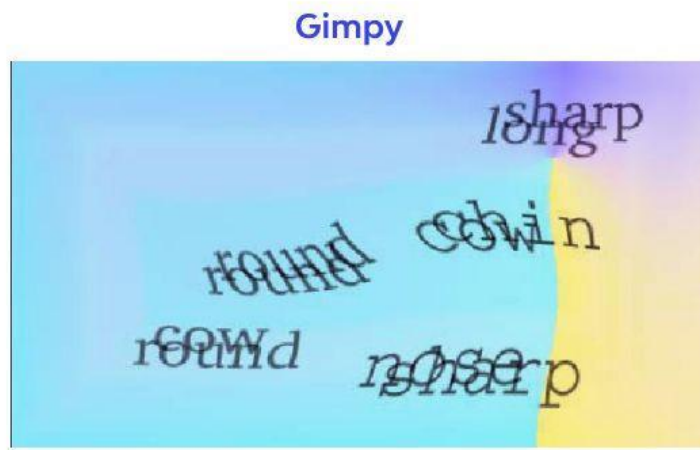
These characters are shown in an odd style by the CAPTCHA, requiring translation. Characters that are scaling, incensed, or turned would all have the option to be coordinated with malevolence. It may moreover join suitable segments like tone, foundation wobbliness, lines, winds, or spots just as covering characters. Despite the way that it may be difficult to understand for individuals, this opening plans for bots doing lacking substance affirmation computations.

Strategies for making text-based CAPTCHAs

Strategies for making text-based CAPTCHAs include:

- 1–1 Gimpy

The gimpy **selects** a handful of emotionally charged words from a **list of 850 words** and **conveys** them in an **unusual font style**.

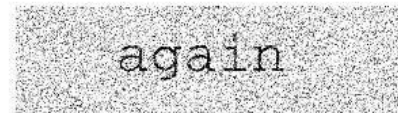
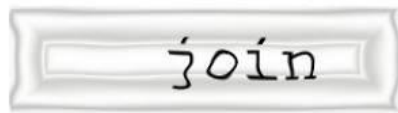


Strategies for making **text-based** CAPTCHAs

1-2. EZ-Gimpy

It's an assortment of Gimpy that just utilizes a **single word**.

EZ-Gimpy

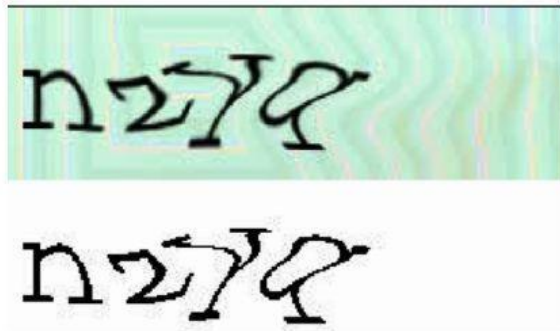


Strategies for making **text-based** CAPTCHAs

1-3. Gimpy-r

This picks **group of letters**, then **bends** يضيف انحاء and **decorates** them with **noise content**.

Gimpy-r



Strategies for making **text-based** CAPTCHAs

1-4. Simard's HIP

This method picks alphabets and numerals at random and then alters them with curves and shadings.

Simard's HIP



Strategies for making text-based CAPTCHAs

2-Image CAPTCHA

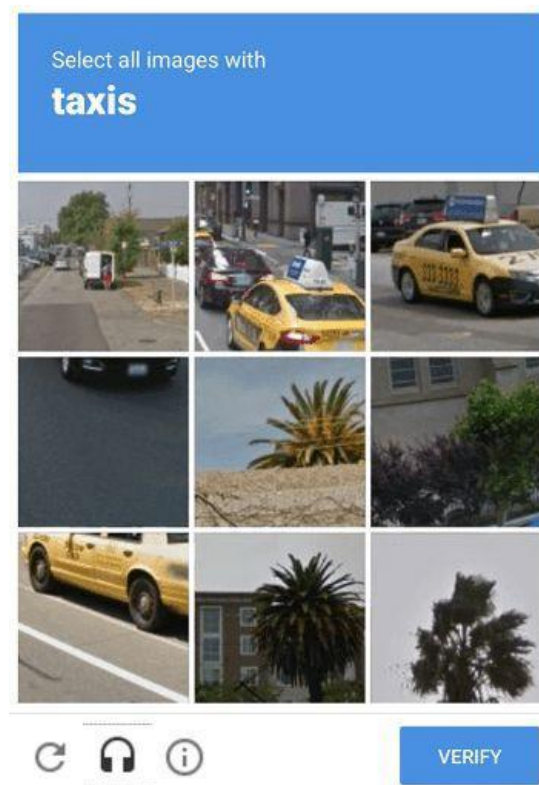
a few photographs of ordinary things are compared. The customer should know which images has a **specific feature** among a **group** of images.

Google, then again, utilizes Google Street View CAPTCHAs that expect clients to enter a street address or a road sign into the material box.

Most clients can address an image based CAPTCHA very quickly.

Regardless, a PC program's capacity to acquire an addressed picture, then, at that point request it's anything but, and afterward work out near one is restricted partly.

Thusly, **picture-based CAPTCHAs give preferred security over text-based cycles.**

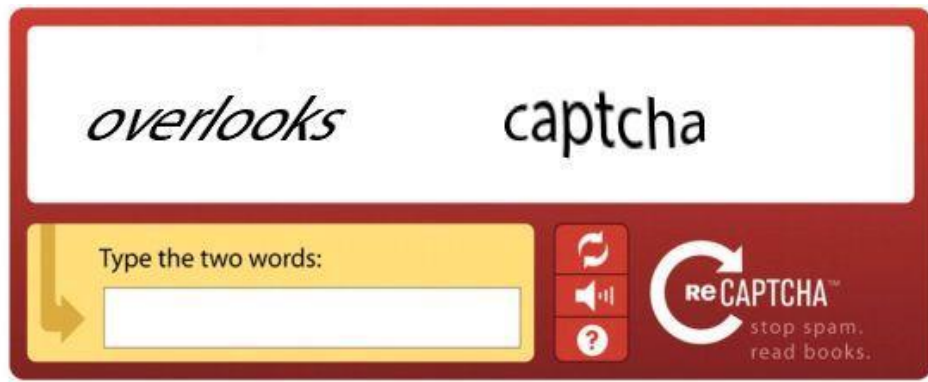


Strategies for making text-based CAPTCHAs

3- Audio CAPTCHA

Audio CAPTCHAs were developed as an **alternative** that grants **accessibility** to **visually impaired** users. These CAPTCHAs are often used in combination with text or image-based CAPTCHAs. **Audio CAPTCHAs present an audio recording** of a **series** of **letters** or numbers which a user then enters.

These CAPTCHAs rely on bots not being able to distinguish relevant characters from background noise. Like text-based CAPTCHAs, these tools can be difficult for humans to interpret as well as for bots.



Strategies for making **text-based** CAPTCHAs

4- Math or Word Problems

4-1. Math Problem:

Some CAPTCHA mechanisms ask users to solve a **simple mathematical problem** such as “3+4” or “18-3”. The assumption is that a bot will find it difficult to identify the question and devise a response.

4-2. Word Problem:

Another variant is a **word problem**, asking the user to type the missing word in a sentence, or complete a sequence of several related terms. These types of problems are accessible to vision impaired users, but at the same time they may be easier for bad bots to solve.

CAPTCHA

This question is to prevent automated spam submissions.

Math question: *

4 + 14 =

Solve this math problem and enter the result. E.g. for 1+3, enter 4.

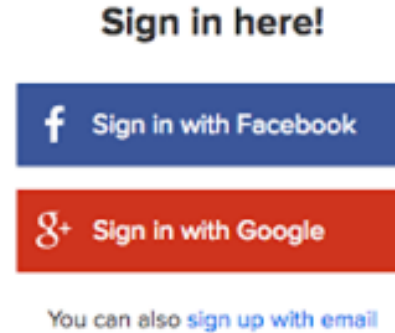
Strategies for making text-based CAPTCHAs

5- Social Media Sign In

A popular alternative to CAPTCHA is requiring users to sign in using a social profile such as Facebook, Google or LinkedIn.

The user's details will be automatically filled in using single sign on (SSO) functionality provided by the social media website.

This is still disruptive, but may actually be easier for the user to complete than other forms of CAPTCHA. An additional benefit is that it is a convenient registration mechanism.



Username or Email Address

Password

☐ Remember Me

Log In

or

Login with Google	
Login with Facebook	
Login with LinkedIn	

Strategies for making text-based CAPTCHAs

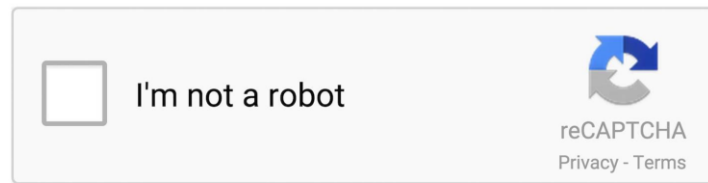
6- No CAPTCHA ReCAPTCHA

This type of CAPTCHA, known for its use by Google, is much easier for users than most other types.

It provides a checkbox saying “I am not a robot” which users need to select – and that’s all.

It works by tracking user movements and identifying if the click and other user activity on the page resembles human activity or a bot.

If the test fails, reCAPTCHA provides a traditional image selection CAPTCHA, but in most cases the checkbox test suffices to validate the user.



How attackers defeat CAPTCHAs

The existing breed of cybercriminals and hackers is so skilled that they will have no hard time bypassing a CAPTCHA. As long as techniques used to bypass CAPTCHA are concerned, most hackers take the help of (Machine Learning) ML algorithms/ services to defeat CAPTCHAs. The algorithm can predict the CAPTCHA pattern and empower a bot to bypass CAPTCHA.

Other than this, the deep learning model is another viable weapon for hackers. The model involves pre-installing a huge number of CAPTCHAs and solving it beforehand. This is done to predict the pattern and learn ways to decode them.

Some techniques to **Bypass** تجاوز **CAPTCHA**

1. Rotate IPs

If many requests come from the same IP address, websites detect it as bot activity and block it. To prevent that, rotate your IPs to scrape without interruptions.

You have to create a pool of proxies and programmatically rotate them to change your IP address per request.

Example

```
# import the required libraries
import requests
import itertools

# create a proxy list (create your own proxy list)
proxy_list = [
    {
        "http": "http://27.64.18.8:10004",
        "https": "http://27.64.18.8:10004",
    },
    {
        "http": "http://161.35.70.249:3128",
        "https": "http://161.35.70.249:3129",
    },
    # ...
]

# define a proxy rotator using a generator
def proxy_rotator(proxy_list):
    return itertools.cycle(proxy_list)

# create a generator from the proxy rotator function
proxy_gen = proxy_rotator(proxy_list)

# rotate the proxies for three requests
for request in range(3):
    # send a request to httpbin.io
    response = requests.get("https://httpbin.io/ip", proxies=next(proxy_gen))
    # print the response text to see your current IP
    print(response.text)
```

Some techniques to **Bypass** تجاوز **CAPTCHA**

2. Rotate **User Agents** (client)

Rotating the User Agent header is another way to prevent CAPTCHAs from appearing while scraping. The User Agent is a string sent with every request. It identifies the browser/HTTP client and operating system of the request source.

Example

```
# import the required Libraries
import requests
import itertools

# create a User Agent List
user_agent_list = [
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.0.0 Safari/537.36",
    "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.0.0 Safari/537.36",
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:128.0) Gecko/20100101 Firefox/128.0",
    # ...
]

# define a User Agent rotator
def rotate_ua(user_agent_list):
    return itertools.cycle(user_agent_list)

# create a generator instance
user_agent_generator = rotate_ua(user_agent_list)

# rotate the User Agent for 4 requests
for request in range(4):
    # send a request to httpbin.io
    response = requests.get(
        "https://httpbin.io/user-agent",
        headers={"User-Agent": next(user_agent_generator)},
    )
    # print the response text to see the current User Agent
    print(response.text)
```

Some techniques to **Bypass** تجاوز

CAPTCHA

3. Simulate Human Behavior

Accurately simulating human behavior is essential to bypass CAPTCHA while scraping a website. For instance, making multiple requests within a few **milliseconds** can result in a **rate-limited IP ban**.

One way to mimic human behavior is to **add delays between requests to reduce your request frequency**. You can **randomize** the delays to make it more **intuitive**. Another approach is implementing exponential backoffs to increase the wait time after each failed request.

The code below shows how to wait between requests and randomize the wait intervals using Python's time and random modules.

Example

```
import time
import random
import requests

# make a GET request to the given URL and print the status code
def make_request(url):
    response = requests.get(url)
    print(f"Request to {url} returned status code: {response.status_code}")

# List of URLs to request
urls = [
    "https://www.scrapingcourse.com/ecommerce/page/1/",
    "https://www.scrapingcourse.com/ecommerce/page/2/",
    "https://www.scrapingcourse.com/ecommerce/page/3/",
]

# range for random wait time (in seconds)
min_wait = 1
max_wait = 5

# iterate through each URL in the list
for url in urls:
    make_request(url)
    wait_time = random.uniform(min_wait, max_wait)
    print(f"Waiting for {wait_time:.2f} seconds before the next request...")
    time.sleep(wait_time)

print("All requests completed.")
```

Some techniques to **Bypass** تجاوز **CAPTCHA**

4 . Save Cookies

Cookies can be your secret weapon when it comes to web scraping. These small files contain data about your interactions **with a website, including your login status, preferences, and more.**

cookies can be beneficial since they save you the time of logging in again, reducing the risk of getting caught. You can also use cookies to persist or pause a web scraping session and resume later.

Example

```
# import the required library
import requests
import json

# create a session
session = requests.Session()

# set the cookie
response = session.get("https://httpbin.io/cookies/set?cookieName=samplecookies")

# convert the cookies to a dictionary
cookies = requests.utils.dict_from_cookiejar(session.cookies)

# create a dictionary with more information
cookie_info = {
    "cookies": cookies,
    "url": "https://httpbin.io",
    "timestamp": response.headers.get("Date"),
}

# write the cookie information to a JSON file
with open("cookie_info.json", "w") as f:
    json.dump(cookie_info, f, indent=4)
```

How to Keep CAPTCHA Codes Secure?

If, by all means, you wish to employ CAPTCHA to secure a website, you need to learn ways to keep the CAPTCHA code secured. As long as CAPTCHA is protected, the website is protected. We present you with the most common and viable ways to secure CAPTCHA codes.

- **Protect the image by doing random distortion.** Make sure the image should be slightly distorted before it's presented to an end-user. This way, it gains immunity against automated attacks.
- **Use matchless CAPTCHAs** so that hackers can't predict the algorithm you're using or the pattern you're following in CAPTCHA creation. Also, make sure that you don't use the same sort of CAPTCHA for all the websites you own. This increases the predictability that works in the favor of hackers. Deploy an algorithm that can easily change the CAPTCHA, create unique codes, and never follow the same pattern.
- **Never use common expressions or equations**, as hackers won't have a tough time bypassing them. $1+1$, $1*1$, or $1-1$ are not the kind of CAPTCHA that is considered safe and sound. Having such a CAPTCHA is equivalent to having no CAPTCHA.
- **Pay attention to the script security** as it's important to make sure that images are non-readable for computers and bots. Some of the best practices to make it include CAPTCHA answer transfer in the form of plain text only and not using open-source CAPTCHA scripts.
- **Combine multiple techniques for CAPTCHA.** Don't always follow the pattern of reading text or image selection. Try to combine two techniques like text reading and audio recording or image selection and audio description. This makes CAPTCHA complex and secure.
- **Using antivirus software:** It is a good move to secure CAPTCHA codes as the tool will keep malicious content at bay.

References

<https://www.geeksforgeeks.org/nlp/opinion-mining-in-nlp/>